

AFRILANG Stdlib

std/c/cryptaeslite

Français. Bibliothèque AFRILANG 'std/c/cryptaeslite' (niveau complexe, catégorie complexe). Elle expose 8 fonction(s) : aeslLen, aeslUpper, aeslLower, aeslTrim, aeslSplit, aeslJoin, aeslReplace, hash32.

```
'import "std/c/cryptaeslite"' · 'use cryptaeslite'
```

```
- 'aeslLen(s text) → number' - 'aeslUpper(s text) → text' - 'aeslLower(s text) → text' - 'aeslTrim(s text) → text' - 'aeslSplit(s text, delim text) → list text' - 'aeslJoin(parts list text, delim text) → text' - 'aeslReplace(s text, from text, to text) → text' - 'hash32(s text) → number'
```

English. AFRILANG library 'std/c/cryptaeslite' (tier complex, category complex). Exports 8 function(s): aeslLen, aeslUpper, aeslLower, aeslTrim, aeslSplit, aeslJoin, aeslReplace, hash32.

```
'import "std/c/cryptaeslite"' · 'use cryptaeslite'
```

```
- 'aeslLen(s text) → number' - 'aeslUpper(s text) → text' - 'aeslLower(s text) → text' - 'aeslTrim(s text) → text' - 'aeslSplit(s text, delim text) → list text' - 'aeslJoin(parts list text, delim text) → text' - 'aeslReplace(s text, from text, to text) → text' - 'hash32(s text) → number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	8	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/cryptaeslite' (niveau complexe, catégorie complexe). Elle expose 8 fonction(s) : aeslLen, aeslUpper, aeslLower, aeslTrim, aeslSplit, aeslJoin, aeslReplace, hash32.

```
'import "std/c/cryptaeslite"' · 'use cryptaeslite'
```

```
- `aeslLen(s text) → number`
- `aeslUpper(s text) → text`
- `aeslLower(s text) → text`
- `aeslTrim(s text) → text`
- `aeslSplit(s text, delim text) → list text`
- `aeslJoin(parts list text, delim text) → text`
- `aeslReplace(s text, from text, to text) → text`
- `hash32(s text) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptaeslite"
use cryptaeslite
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- aeslLen(s text) → number•
aeslUpper(s text) → text•
aeslLower(s text) → text•
aeslTrim(s text) → text•
aeslSplit(s text, delim text) → list•
aeslJoin(parts list text, delim text) → text•
aeslReplace(s text, from text, to text) → text•
hash32(s text) → number

4. Exemples d'utilisation

```
import "std/c/cryptaeslite"
use cryptaeslite
```

```
create result = aeslLen(/* args */)
say result
```

```
create result = aeslUpper(/* args */)
say result
```

```
create result = aeslLower(/* args */)
say result
```

```
create result = aeslTrim(/* args */)
say result
```

```
create result = aeslSplit(/* args */)
say result
```

```
create result = aeslJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/cryptaeslite"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module cryptaeslite

```
function aeslLen(s text) returns number
end
```

```
function aeslUpper(s text) returns text
end
```

```
function aeslLower(s text) returns text
end
```

```
function aeslTrim(s text) returns text
end
```

```
function aeslSplit(s text, delim text) returns list text
end
```

```
function aeslJoin(parts list text, delim text) returns text
end
```

```
function aeslReplace(s text, from text, to text) returns text
end
```

```
function hash32(s text) returns number
end
end
```

English

1. Overview

AFRILANG library 'std/c/cryptaeslite' (tier complex, category complex). Exports 8 function(s): aeslLen, aeslUpper, aeslLower, aeslTrim, aeslSplit, aeslJoin, aeslReplace, hash32.

```
'import "std/c/cryptaeslite"' · 'use cryptaeslite'
```

```
- `aeslLen(s text) → number`
- `aeslUpper(s text) → text`
- `aeslLower(s text) → text`
- `aeslTrim(s text) → text`
- `aeslSplit(s text, delim text) → list text`
- `aeslJoin(parts list text, delim text) → text`
- `aeslReplace(s text, from text, to text) → text`
- `hash32(s text) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptaeslite"
use cryptaeslite
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- aeslLen(s text) → number•
aeslUpper(s text) → text•
aeslLower(s text) → text•
aeslTrim(s text) → text•
aeslSplit(s text, delim text) → list•
aeslJoin(parts list text, delim text) → text•
aeslReplace(s text, from text, to text) → text•
hash32(s text) → number

4. Usage examples

```
import "std/c/cryptaeslite"
use cryptaeslite
```

```
create result = aeslLen(/* args */)
say result
```

```
create result = aeslUpper(/* args */)
say result
```

```
create result = aeslLower(/* args */)
say result
```

```
create result = aeslTrim(/* args */)
say result
```

```
create result = aeslSplit(/* args */)
say result
```

```
create result = aeslJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/cryptaeslite"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module cryptaeslite

```
function aeslLen(s text) returns number
end
```

```
function aeslUpper(s text) returns text
end
```

```
function aeslLower(s text) returns text
end
```

```
function aeslTrim(s text) returns text
end
```

```
function aeslSplit(s text, delim text) returns list text
end
```

```
function aeslJoin(parts list text, delim text) returns text
end
```

```
function aeslReplace(s text, from text, to text) returns text
end
```

```
function hash32(s text) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/cryptaeslite"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/cryptaeslite/>

Source (excerpt)

```
module cryptaeslite

function aeslLen(s text) returns number
end

function aeslUpper(s text) returns text
end

function aeslLower(s text) returns text
end

function aeslTrim(s text) returns text
end

function aeslSplit(s text, delim text) returns list text
end

function aeslJoin(parts list text, delim text) returns text
end

function aeslReplace(s text, from text, to text) returns text
end

function hash32(s text) returns number
end
end
```